NP-completeness

Part VII Selected Topics

Bit encodings, quantifiers, and verifiable certificates define computational hardness. We construct the core reductions, prove both semantic directions, and bound their size and running time. Prerequisites are decision problems, Turing machines, reductions, and Boolean circuits.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter map

The sections cover polynomial time, polynomial-time verification, reducibility, NP-completeness proofs, and NP-complete problems. The goals are to define P, NP, NP-hardness, and NP-completeness, construct many-one reductions, and prove the SAT, clique, vertex-cover, Hamiltonian-cycle, TSP, and subset-sum connections.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decision languages and encodings

An abstract problem maps instances to answers. A decision problem has yes/no answers, represented by a language $L = \{x : \text{the answer on } x \text{ is yes}\}$. One object can have different encodings. An integer t needs about $\log_2(t+1)$ bits in binary but t symbols in unary. Polynomial time is measured in actual input bits, so the encoding must be specified.

$$L \in \mathsf{P} \iff \exists \text{ deterministic TM } M, \exists \text{ polynomial } p : \forall x,\ M(x) \text{ halts within } p(|x|), \quad M(x) = 1 \iff x \in L.$$

The quantifier is every x, not some x. Malformed encodings must also be rejected or otherwise handled as specified in polynomial time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial composition and practical speed

If A produces a valid encoding within $p_A(n)$ steps and B solves the next problem within p_B steps, their composition takes at most $p_A(n) + p_B(p_A(n))$. Output length cannot exceed production time. Substituting a polynomial into another polynomial remains polynomial.

Thus a problem that reduces in polynomial time to a polynomial-time solvable problem is also in P. Multiple polynomial conversions compose, provided the semantic direction is correct and lengths stay bounded.

Running quickly on one computer is not a membership proof. $O(n^{100})$ is polynomial, while a low-constant exponential algorithm can be more useful for small n. Complexity classes and engineering speed answer different questions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pseudopolynomial is not input-bit polynomial

A cost $O(nt)$ may be exponential in the bit length of t. Subset-sum DP has $n(B+1)$ cells and costs $O(nB)$. With binary B, this depends on numerical magnitude rather than input length and is pseudopolynomial.

For a traceable family, let $n = m, B = 2^m, a_i = 2^{m-1} + i, i = 1, \ldots, m, m \geq 2$. Each number has $\Theta(m)$ bits, and B has m+1 bits. Total input length is $N = \Theta(m^2)$, but full-table work is

$$\Theta(m2^m) = 2^{\Theta(m)} = 2^{\Theta(\sqrt{N})}.$$

If bounded by CN^c for fixed c,C, then eventually $m2^m \leq C'm^{2c}$, contradicting $m2^m/m^{2c} \to \infty$. The algorithm can still be useful for small B or unary input. It does not prove binary SUBSET-SUM belongs to P.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

NP's complete quantifiers

Fast verification requires a bounded certificate and a deterministic verifier:

$$L \in \text{NP} \iff \exists V, \exists \text{ polynomials } p, q: \quad \forall x, \quad x \in L \iff \exists y \ (|y| \leq q(|x|) \wedge V(x, y) = 1).$$

The verifier halts on **all** x,y within $p(|x| + |y|)$ steps. The string y is a certificate or witness. Quantifier order matters. A yes instance has some witness. For a no instance, no witness can make the verifier accept. Nondeterministically guessing a witness represents this existential quantifier as an accepting computation path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Completeness and soundness of a verifier

For relation R(x,y), completeness says every $x \in L$ has a polynomial-length y with R(x,y)=1. Soundness says that existence of an accepted y implies $x \in L$. Both are needed for the equivalence. Completeness alone may accept false instances. Soundness alone may miss true ones.

For Hamiltonian cycle, a certificate is a sequence of n vertices. Verify that its length is n, that it is a permutation of all vertices, that each consecutive pair is an original edge, and that the last vertex connects to the first. Checking only the length is unsound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P is contained in NP

If $L \in P$, let M be its deterministic polynomial-time decider. Define V(x,y) to ignore y and run M(x). Use the empty certificate. If x is in L, that certificate accepts. If x is outside L, every y is rejected. Thus soundness and completeness hold and $P \subseteq NP$.

The reverse containment remains open. Failure to find a polynomial algorithm is not a proof that none exists.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial many-one reductions

The notation $A \leq_p B$ means there is a function f computable in polynomial time on every source instance, producing a valid target instance, such that for every x,

$$x \in A \iff f(x) \in B.$$

The direction converts source A into target B. To prove a new B hard, reduce a known hard A to B. Reducing B to an easy problem cannot establish hardness.

A many-one reduction produces one B instance. It does not adapt the source after a solver response or make multiple oracle calls, unless those computations themselves fit inside the legitimate polynomial-time conversion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prove both semantic directions

The forward direction $x \in A \Rightarrow f(x) \in B$ constructs a target yes witness from each source yes witness. The reverse direction $f(x) \in B \Rightarrow x \in A$ shows that any target witness decodes to a source yes instance. Authors sometimes swap the names completeness and soundness depending on viewpoint, so state the actual implication.

For a clique reduction, any target k-clique must carry enough consistent information to reconstruct a satisfying assignment. A gadget, edge rule, or threshold that permits contradictory assignments breaks the reverse implication.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Transitivity with length bounds

Suppose $A \leq_p B$ via f and $B \leq_p C$ via g. Define $h(x) = g(f(x))$. If x belongs to A, f(x) belongs to B and h(x) to C. Conversely, h(x) in C implies f(x) in B and then x in A. Thus $x \in A \iff h(x) \in C$.

If f takes $p_f(n)$ time, then $|f(x)| \leq p_f(n)$. Composition takes at most $p_f(n) + p_g(p_f(n))$, a polynomial. Therefore $A \leq_p C$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-hard and NP-complete

A language L is NP-hard if $A \leq_p L$ for every $A \in NP$. It is NP-complete if it is both NP-hard and in NP.

Given known NP-complete C, prove L is in NP and $C \leq_p L$. For any A in NP, hardness of C gives $A \leq_p C$, and transitivity gives $A \leq_p L$. Without membership, the conclusion is only NP-hard. Without the reduction in this direction, known hardness has not been transferred.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The first NP-complete problem

CIRCUIT-SAT asks whether a finite acyclic Boolean circuit with fan-in-two AND/OR gates and unary NOT gates has an input making its designated output 1. Original input bits form the certificate. Evaluate gates in topological order and check the output in linear gate-and-wire time, proving NP membership.

For hardness, fix any L in NP, its deterministic verifier V, certificate bound q(n), and time bound. Given x, the free inputs to circuit C_x must represent witness y. They are not an unspecified accepting state of one fixed computation.

Normalize witnesses to exactly $q(n) + b(n)$ bits, where $b(n) = 1 + \lceil \log_2(q(n) + 1) \rceil$. The first b bits encode actual length ℓ; the next q bits store y padded with zeros. The new verifier checks $0 \leq \ell \leq q(n)$, reads only those ℓ witness bits, then simulates V. If q(n)=0, the one-bit length field permits only zero. This preserves existence of a witness without conflating different string lengths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expand local machine transitions into gates

Fix a single-tape deterministic TM simulating that verifier, remaining in its terminal state after acceptance or rejection. A multitape-to-single-tape simulation costs at most a polynomial factor. Let T(n) be its polynomial step bound, including the normalized witness. Initial tape length is $N = O(n + q(n) + 1)$, including the nonempty length field.

A head moves at most one cell per step. Represent $O(N + T)$ cells and retain T+1 blank cells beyond both ends. A cell label contains its tape symbol and whether the head is there, with its state if present. Fixed machine states and alphabet use a constant b bits per label.

The label at time t+1 and position j depends only on labels j-1,j,j+1 at time t. Each output bit is a Boolean function of a fixed 3b inputs. Express its truth table as an OR of minterms, each an AND of inputs or negated inputs, then replace large gates by fan-in-two gates. Because the machine and b are fixed, every local update uses $O(1)$ gates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circuit size and the simulation equivalence

The initial layer hardwires x, blanks, head location, and initial state. Only normalized witness bits are free. For every time $0, \ldots, T-1$ and cell, copy the fixed local circuit and connect it to the next layer. An OR tree checks for an accepting-state label at time T.

Gate and wire counts are $O(T(N+T))$. Gate indices use $O(\log(T(N+T)))$ bits. Enumerating times, cells, and a fixed truth table produces the circuit in polynomial time, with no hidden exponential hardware.

For each y, induction on t proves every layer is the actual configuration of V(x,y). The initial layer is legal, and local transitions produce exactly the next configuration. Invalid local labels are never reached. Terminal acceptance persists, so $C_x(y) = 1$ iff V accepts.

If x is in L, an accepting witness satisfies the circuit. If the circuit is satisfiable, its free witness bits make V accept, so x is in L. Every NP language reduces this way to CIRCUIT-SAT, proving NP-completeness.

Circuit-to-SAT gate constraints

Give each wire value a variable, sharing the same gate-output variable where needed. Require output z_{out} by a unit clause. Encode every gate equivalently:

Relation	CNF constraints
z=u AND v	$(\neg z \vee u) \wedge (\neg z \vee v) \wedge (z \vee \neg u \vee \neg v)$
z=u OR v	$(z \vee \neg u) \wedge (z \vee \neg v) \wedge (\neg z \vee u \vee v)$
z=NOT u	$(z \vee u) \wedge (\neg z \vee \neg u)$

For AND, the first two clauses force both inputs true if z is true; the last forces z true if both inputs are true. OR and NOT follow by the same two-direction truth cases. Fix constant wires with unit clauses.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circuit-to-SAT gate constraints (continued)

An accepting circuit input extends to all computed wire values and satisfies the constraints. Conversely, gate equivalences force every wire to equal its actual value in topological order, and the output unit clause forces acceptance. At most three clauses per gate give linear total size in gates and wires. Repeatedly expanding shared subcircuits into formula trees could instead cause exponential growth.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SAT to three-variable clauses

Parse a formula into unary NOT and binary AND/OR nodes. Replace implication by $\neg a \vee b$. Other fixed-arity connectives have fixed-size truth-table gadgets. Give each internal node a new variable, apply the gate clauses, and require the root true. Remove repeated literals within a clause. Delete clauses containing both a literal and its negation, since they are tautologies.

Clauses now have at most three literals. To require exactly three distinct variables per clause, use fresh padding variables for each clause:

- Replace $(a \vee b)$ by $(a \vee b \vee p) \wedge (a \vee b \vee \neg p)$. If the original is true, any p works. If false, the pair forces both p and its negation.
- Replace unit a by the four clauses $(a \vee p \vee q), (a \vee p \vee \neg q), (a \vee \neg p \vee q), (a \vee \neg p \vee \neg q)$. If a is false, every p,q assignment falsifies one clause. If true, all pass.
- An empty clause is false, so output the padded version of $z \wedge \neg z$. An empty conjunction is true, so output one fixed satisfiable three-variable clause.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

SAT to three-variable clauses (continued)

Original assignments extend through parse-node values. Conversely, the tree constraints recover node semantics. Padding preserves each clause requirement. This is equisatisfiability, not literal equality of functions after adding variables. Constant clauses per node and at most four per padding step give polynomial size and construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splitting a long clause

For $l_1 \vee \cdots \vee l_r$, $r > 3$, a chain also works:

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

If all original literals are false, the first clause forces y1 true, each middle clause forces the next y true, and the last forces the final y false, a contradiction. If some literal is true, let yj indicate that the first j+1 literals are all false. Every chain clause then holds. This uses r-3 new variables and r-2 clauses. Unrestricted distributive expansion of a formula into CNF may be exponentially large.

For $F = (a \vee b) \wedge \neg a$, set $u = a \vee b, v = \neg a, z = u \wedge v$, add their gate clauses, and require z. Input a=false,b=true extends to u=v=z=true. If a=true, the NOT gadget forces v=false while z and AND require v=true, excluding a false acceptance.

The book's three-CNF definition uses three distinct literals. The main reductions here can further normalize to three distinct variables. Deduplication and padding also handle repeated literals before the occurrence construction.

Three-CNF-SAT to CLIQUE

For a formula F with k clauses, create one vertex per literal occurrence and set the clique threshold to k. For k=0, the empty conjunction is true and maps separately to `(empty graph,0)`. Otherwise k≥1.

Vertices within a clause have no edges. Vertices from different clauses are adjacent exactly when their literals are not complementary. With three occurrences per clause there are 3k vertices.

Let |F| be actual binary encoding length. Intern appearing variable names into n IDs, n≤3k. Checking vertex pairs costs $O(k^2)$ word comparisons. Bit accounting includes $O(\log(n+k))$ bits per ID comparison and edge endpoint. A safe bound is $O(|F| \log |F| + k^2 \log(n+k))$. An adjacency matrix has $\Theta(k^2)$ bits. Comparing arbitrarily long variable names is not automatically constant time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A consistent occurrence clique

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Assignment $x_1 = false, x_2 = false, x_3 = true$ satisfies all clauses. Select x3 from the first, ¬x1 from the second, and x3 from the third. They come from different clauses and are pairwise noncomplementary, forming a size-3 clique.

This is not the only witness. Selecting x1 from the first and ¬x1 from the second would fail because they lack an edge. Under the chosen assignment, x1 is false anyway. The graph cannot combine contradictory requirements into a witness.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

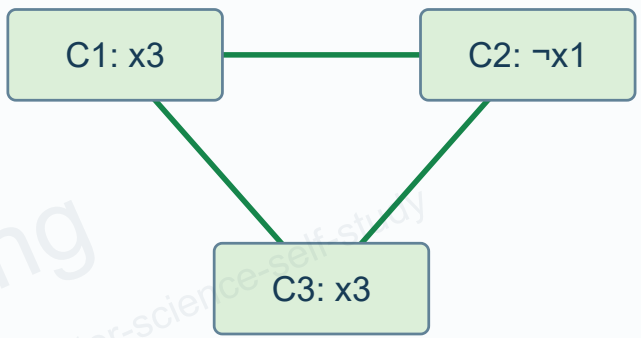
A consistent occurrence clique (continued)

Connect different clauses only for noncomplementary literals

C1 / C2	¬x1	x2	¬x3
x1	×	✓	✓
¬x2	✓	×	✓
x3	✓	✓	×

✓ Edge; × complementary pair, no edge

The three selected occurrences are consistent



x1=0, x2=0, x3=1: one true literal from each group

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both directions of the clique reduction

If F is satisfiable, fix a satisfying assignment and select one true occurrence from every clause. Two chosen literals cannot be complementary under one assignment. Every pair therefore has an edge, producing a k-clique.

Conversely, a k-clique has at most one vertex per clause since same-clause pairs lack edges. With k clauses, it selects exactly one from each. Every selected pair is noncomplementary, so no variable is required both true and false. Set each selected variable to satisfy its chosen literal and set unselected variables arbitrarily. These consistent choices satisfy every clause.

Thus F is satisfiable iff the constructed graph has a k-clique. A list of k vertices is a polynomial certificate checked by pairwise edges, so CLIQUE is NP-complete.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Clique, independent set, and vertex cover

An independent set has no edge between its members. A vertex cover touches every edge. For a loopless undirected n-vertex graph, its complement has edge uv exactly when u≠v and G lacks that edge. Thus a set is a clique in G iff it is independent in its complement.

If S is independent in H, every H edge has an endpoint outside S, so $V \setminus S$ is a vertex cover. Conversely, a cover $V \setminus S$ forbids any edge with both endpoints in S, so S is independent. For $0 \le k \le n$,

$$\mathrm{CLIQUE}(G, k) \iff \mathrm{VC}(\overline{G}, n - k).$$

The reductions $(G, k) \mapsto (\overline{G}, n - k)$ and $(H, \ell) \mapsto (\overline{H}, n - \ell)$ both take $O(n^2)$ pair checks. A clique of at least k contains one of exactly k. A cover of at most n-k can be enlarged to exactly n-k. Handle out-of-range thresholds with fixed yes/no instances rather than negative certificate sizes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A complement example

For vertices {1,2,3} and edges {12,23}, the set {1,2} is a size-2 clique. The complement has only edge 13. The same set is independent there, and its complement {3} covers the only edge, with size 3-2=1.

The transformation is to VC on the complement, with threshold n-k. Complementation is polynomial and reversible, giving reductions in both directions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex cover to Hamiltonian cycle

A Hamiltonian-cycle certificate is a permutation of all vertices with valid consecutive edges and closure. This proves NP membership. For hardness, build an undirected gadget for every original edge {u,v}. Crossings in its drawing are not vertices, and no planar crossing gadget is required.

Create twelve vertices a1,...,a6 and b1,...,b6, where ai=[u,v,i] and bi=[v,u,i]. Add fourteen internal edges. Ten form the two chains a1-a2-a3-a4-a5-a6 and b1-b2-b3-b4-b5-b6. Four cross links are a1-b3, a3-b1, a4-b6, a6-b4.

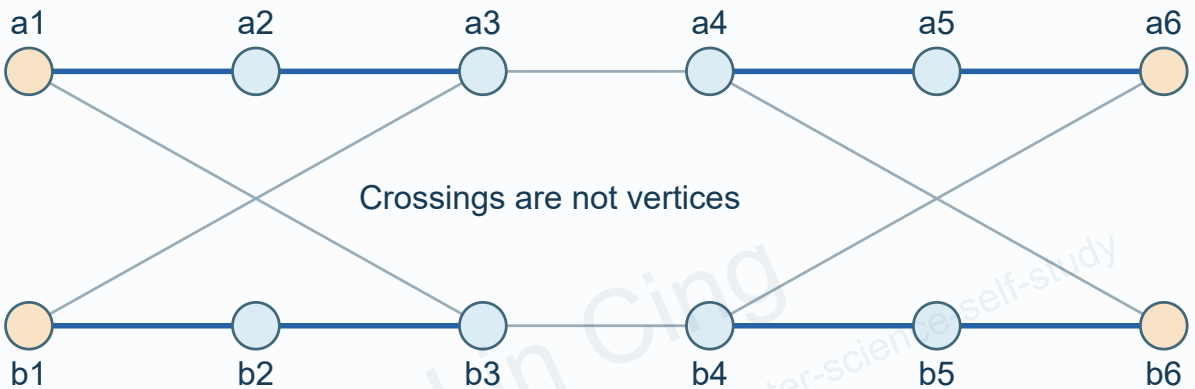
Only a1,a6,b1,b6 may connect outside the gadget. The other eight vertices need two internal edges in any final cycle. In particular a2,a5,b2,b5 already have degree two in the gadget, forcing their eight adjacent chain edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex cover to Hamiltonian cycle (continued)

HC gadget: only a1,a6,b1,b6 connect outside

a_i=[u,v,i]; b_i=[v,u,i]



Eight blue edges are forced by degree 2 at a2,a5,b2,b5. Six gray edges remain undecided.

Middle edges a3−a4 and b3−b4 force the four cross links. Orange marks the four ports.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three legal gadget covers

The remaining choices are the middle edges a3-a4 and b3-b4. Once these are selected or omitted, degree two at internal vertices forces every cross link.

Middle edges selected	Cover of all twelve vertices
Both	Two disjoint six-vertex paths, a1 through a6 and b1 through b6
Only b3-b4	One path a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6
Only a3-a4	The symmetric full path from b1 to b6
Neither	Two closed six-vertex cycles with no way to join the external cycle, forbidden

These cases exhaust all internal degree configurations. A legal Hamiltonian cycle uses one of the first three. It cannot enter an a port and leave through a b external chain. It must cover every gadget vertex. Locally, at least one endpoint of the original edge must be selected.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Middle edges force external port use

Let x indicate selection of a3-a4 and y selection of b3-b4. Vertex a3 already uses forced edge a2-a3, so a3-b1 has indicator 1-x. Similarly a4-b6 has 1-x. Edges a1-b3 and a6-b4 each have 1-y.

Thus a1,a6 have internal degree 2-y and external degree y. Ports b1,b6 have external degree x. Both ends on one original-vertex side connect externally together or neither does.

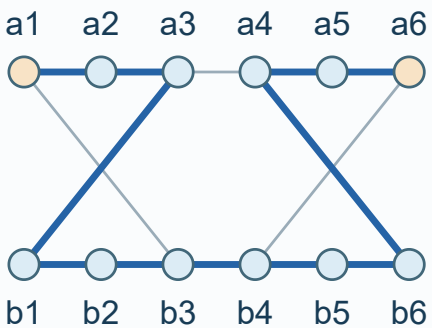
For x=y=0, the cycles are a1-a2-a3-b1-b2-b3-a1 and a4-a5-a6-b4-b5-b6-a4. Every vertex already has degree two, so neither cycle connects to a selector. They cannot form the whole graph's single Hamiltonian cycle. The three remaining full path covers are shown below.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Middle edges force external port use (continued)

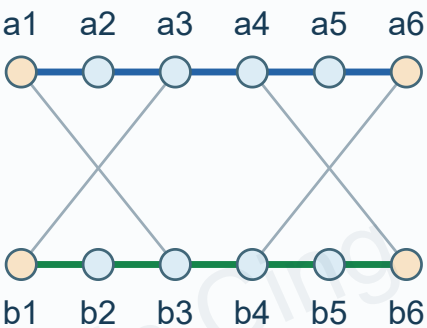
All three legal modes preserve the original-vertex side

u side only: 12 vertices



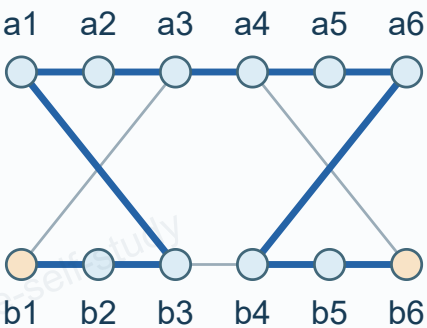
x=0, y=1

6 vertices on each side



x=1, y=1

v side only: 12 vertices



x=1, y=0

Gray edges are unused. Blue/green show paths. Each orange port needs one external edge for degree 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Incident chains and selectors

VERTEX-COVER asks for size at most k. Remove isolated vertices without changing the minimum cover. If k<0, or k=0 with an edge present, output a fixed no Hamiltonian-cycle instance. If no edges remain, or k is at least the remaining vertex count, output a fixed yes triangle. The main case has no isolated vertices and $1 \leq k < |V|$. A smaller cover can be enlarged to exactly k nonisolated vertices.

For each original u, order its neighbors u(1),...,u(d) arbitrarily. Connect its gadget sides by edges $[u, u(i), 6] - [u, u(i+1), 1]$, i=1,...,d-1. An external path selecting u must follow all of u's incident gadgets from one chain endpoint to the other.

Add k selectors s1,...,sk. Each selector connects to both ends of every u chain, with no other external edges. There are $12|E| + k$ vertices, $14|E|$ internal edges, at most $2|E|$ chain links, and $2k|V|$ selector links. The size is polynomial. Both source and target graphs are undirected.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Both directions of vertex cover to cycle

Given a cover $C = \{u_1, \ldots, u_k\}$, make one path through each selected vertex's incident chain. If only one endpoint of an original edge is selected, use that side's full twelve-vertex path. If both endpoints are selected, use the two disjoint six-vertex paths. Every edge has a selected endpoint, so every gadget vertex is visited exactly once. Join s1, the u1 chain, s2, the u2 chain, ..., sk, the uk chain, and back to s1. This visits every selector once and gives one Hamiltonian cycle.

Conversely, remove the k selectors from a Hamiltonian cycle. This leaves k disjoint paths. Local gadget modes forbid switching original-vertex identity, so each path follows one original u's entire incident chain. Two paths cannot select the same u, since they would share a chain endpoint or gadget vertex. Select these k original vertices. Every gadget is visited, and every legal mode uses at least one endpoint's chain. Hence every original edge is covered.

Therefore VC(G,k) holds iff the constructed graph has a Hamiltonian cycle. Construction, decoding, and size are polynomial. Fixed yes/no instances handle the preliminary cases without inventing empty chains for selectors.

A full twenty-five-vertex cycle

For original path u-v-w with k=1, choose cover {v}. Build U=W_uv and V=W_vw, each with twelve vertices, and one selector. The b side of U and a side of V represent v. Connect U.b6 to V.a1 and the selector to the two ends of the v chain.

Use these full paths, with every point prefixed by its gadget:

- U: b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6.
- V: a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6.

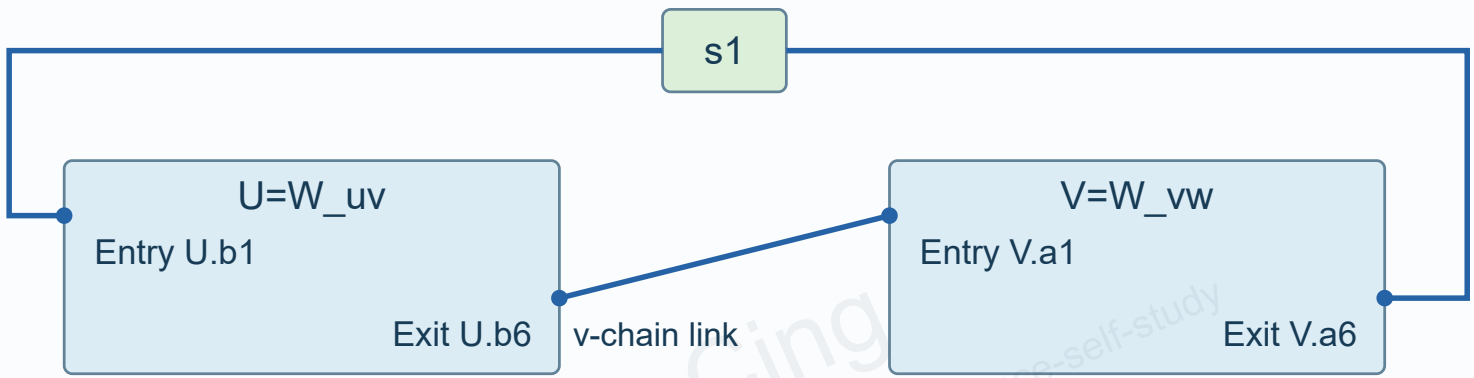
The complete cycle starts s1→U.b1, follows all of U, takes U.b6→V.a1, follows all of V, and returns V.a6→s1. Same-named vertices in different gadgets are distinct. Exactly 24 gadget vertices plus one selector are visited once. Every adjacency is an internal edge, chain link, or selector edge.

Both original edges have only v selected, so both gadgets use a full path on v's side. Choosing only u would leave W_vw uncovered and cannot create a cycle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A full twenty-five-vertex cycle (continued)

Original path u−v−w, cover={v}: one selector closes a 25-vertex cycle



Each box contains the full 12-vertex path listed in the notes. U and V vertices are distinct.

This shows the selected HC. All external edges follow the selector and incident-chain rules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hamiltonian cycle to TSP decision

Given an undirected graph with $n \geq 3$, build complete graph K_n. Give original edges weight 1 and missing edges weight 2. Set threshold B=n. Construction takes $O(n^2)$.

A Hamiltonian cycle uses n original edges and becomes a tour of cost n. Conversely, every tour has n edges of weight at least 1. Total cost at most n forces every edge to have weight 1, so every tour edge belongs to the original graph. It is a Hamiltonian cycle there.

Thus HC(G) iff TSP(Kn,w,n). TSP decision is in NP. Its certificate is a vertex permutation, checked for uniqueness, closure, and total cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TSP trace and small graphs

For a triangle with edges 12,23,31, all three weights are 1. Tour $1 \to 2 \to 3 \to 1$ costs 3 and meets threshold $B=3$. Removing original edge 31 leaves that edge in the complete target graph with weight 2. Every three-point tour then costs $1+1+2=4>3$, matching the absence of an original Hamiltonian cycle.

Simple undirected graphs with $n<3$ have no Hamiltonian cycle under this chapter's convention. Map them to a fixed no TSP instance. The n-edge argument above assumes $n \geq 3$.

Optimization TSP returns a minimum cost and, when requested, a tour. A polynomial exact optimizer could decide the threshold problem by comparison, so hardness of decision also makes exact optimization hard unless P=NP.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decision, optimization, and search

Optimization counterparts include minimum-cost TSP, minimum SET-COVER, and maximum CLIQUE. An exact polynomial optimizer solves the decision version by comparing its value to the threshold. In the other direction, an exact decision oracle can find an integer optimum by binary search if upper and lower bounds have polynomial bit length and every threshold query is answered correctly.

Finding the optimum value does not itself output a witness. Reconstruction needs additional queries, for example fixing decisions step by step. A known value cannot be presented as an already constructed optimal solution.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decision, optimization, and search (continued)

Classification	Meaning	Consequence	Does not imply
NP-hard language L	Every NP language reduces to L	If L is in P, P=NP	L is in NP
NP-complete language L	L is in NP and NP-hard	Short certificates and universal NP reductions	An optimization program is automatically a language
CLIQUE decision	Ask whether a k-clique exists	Membership and hardness proved here	Output of a maximum clique
MAX-CLIQUE optimization	Return maximum size, optionally a witness	An exact polynomial solver decides CLIQUE	Direct application of a language definition to an optimization output type

NP-hardness concerns reductions. NP-completeness also requires membership. Decision answers yes/no, optimization returns a value, and search returns a witness. Any claimed equivalence must explain how one output supplies the other's required answer.

Subset-sum membership and dynamic programming

Decision SUBSET-SUM asks whether some subset of given positive integers sums exactly to target t. A certificate lists selected indices. Check bounds, nonrepetition, and exact sum in polynomial time, proving membership in NP.

Let dp[i,s] mean that a subset of the first i integers reaches s. Then

$$dp[i, s] = dp[i - 1, s] \vee (a_i \leq s \wedge dp[i - 1, s - a_i]).$$

The $O(nt)$ algorithm is pseudopolynomial with binary t. The reduction below uses large values with polynomial bit length, consistently with that DP limitation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-CNF-SAT to a digit table

Normalize to n appearing variables and k clauses, each with three distinct variables and no complementary tautology. Deduplication and padding above achieve this. Map constant true or false formulas separately to fixed yes/no integer instances.

Create n+k decimal columns, first for variables and then clauses. These columns compactly define integers. The final delivered encoding is binary.

- For each xi, create Ti and Fi. Both have 1 in variable column i and 0 in other variable columns. Ti has 1 in each clause containing xi; Fi has 1 in each clause containing ¬xi. Other clause digits are zero.
- For each clause j, add two slack numbers, with digits 1 and 2 respectively in that clause column and zero elsewhere.
- The target has 1 in each variable column and 4 in each clause column.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three-CNF-SAT to a digit table (continued)

There are 2n+2k positive numbers, each with n+k decimal digits and $O(n+k)$ binary bits. Total output length is $O((n+k)^2)$, and construction is polynomial. If the SUBSET-SUM input format requires a set rather than a multiset, the nondegenerate premises ensure Ti and Fi differ: each variable appears, and positive/negative occurrences cannot coincide in a clause. Different variables have different high columns, and slack columns also distinguish the slack numbers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

No carry for every candidate subset

An arbitrary subset sums each variable column to at most 2. A clause has three distinct literals, so all truth-value numbers together contribute at most three 1s in its column. Slack adds at most 1+2=3. Thus every clause sum is at most 6<10. **No subset can carry**, not only a correctly constructed witness.

If the formula is satisfiable, choose exactly Ti or Fi according to each variable's value. Variable columns become 1. Each clause has r true literals, r=1,2,3. Fill its deficit 4-r by slack 1+2, 2, or 1, respectively. All columns reach the target.

Conversely, any subset reaching the target has exactly one of Ti,Fi per variable, because no carries can alter its variable column. These choices define a consistent assignment. Each clause target is 4 but its slack contributes at most 3, so some chosen truth-value number contributes 1 there. The corresponding literal is true. Every clause is satisfied.

The two implications and polynomial size prove NP-hardness. Together with NP membership, SUBSET-SUM is NP-complete.

A hand-checkable no-carry sum

Use $F = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$. Column order is x1,x2,x3,C1,C2.

Numbers	Decimal column representation
T1,F1	10010, 10001
T2,F2	01011, 01000
T3,F3	00110, 00101
S1,S1′	00010, 00020
S2,S2′	00001, 00002
Target	11144

Assignment (true,true,false) chooses T1+T2+F3=11122. Each clause has two true literals. Add S1′=20 and S2′=2 to reach 11144. No carries occur. Each high digit 1 confirms one truth-value number per variable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A hand-checkable no-carry sum (continued)

Values can be of order 10^{n+k}, while binary length is only $O(n+k)$. The reduction has polynomial output even though exact $O(nt)$ DP is not polynomial in bit length.

T1+T2+F3=11122; add slack 00022 to reach target 11144

	x1	x2	x3	C1	C2
T1	1	0	0	1	0
T2	0	1	0	1	1
F3	0	0	1	0	1
slack	0	0	0	2	2
target	1	1	1	4	4

Each variable digit is 1

Each clause has 2 true literals

Slack 2 fills each to 4

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Why universal no-carry matters

Proving no carry only for satisfying-assignment selections would not establish the reverse direction. A wrong subset might otherwise forge target digits through carries. Here every subset has variable digits at most 2 and clause sums at most 6, so no carry can occur for any candidate.

Converting an n+k digit decimal number to binary by repeated multiplication by 10 and digit addition has a simple $O((n+k)^2)$ bit implementation. Performing it for all 2n+2k numbers is still polynomial. Verification sums at most $O(n+k)$ numbers of $O(n+k)$ bits, adding only $O(\log(n+k))$ bits to the sum.

For assignment (false,false,false) in the example, the two clauses have 0 and 2 true literals. Even both slack numbers give only 3 in the first clause column, below 4. No carry from another column can conceal the failure.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Encoding invariance, closure, and coNP

If two encodings convert to each other in polynomial time, P membership is unchanged. Convert then decide; conversion bounds output length, and polynomial composition applies. Binary and unary integers lack this two-way guarantee. Writing 2^{b-1} in unary from b binary bits requires exponential output.

P is closed under complement, union, and intersection by negating or combining polynomial deciders. For concatenation L1L2, test all n+1 split positions with the two deciders. For L*, use reachable[0]=true and test substrings x[i+1..j] for i<j as possible next pieces. $O(n^2)$ polynomial tests plus DP decide reachable[n]; accept the empty string directly. Both operations preserve P.

Define $coNP = \{L : \overline{L} \in NP\}$. It describes short certificates for no instances, not languages outside NP. Neither P=NP nor NP=coNP is known. If an NP-complete C belongs to coNP, take any A in NP with $A \leq_p C$. The same map gives $\overline{A} \leq_p \overline{C}$. Compose the reduction with the NP verifier for $\overline{C}$ to obtain $\overline{A} \in NP$, so NP⊆coNP. Taking complements yields equality. This is a conditional implication, not a proved separation.

Reduction chains and hypotheses

The main chain is CIRCUIT-SAT $\leq_p$ SAT $\leq_p$ 3-CNF-SAT $\leq_p$ CLIQUE. Complementation connects CLIQUE, INDEPENDENT-SET, and VERTEX-COVER. Another chain is VERTEX-COVER $\leq_p$ HAMILTONIAN-CYCLE $\leq_p$ TSP decision. The no-carry construction gives 3-CNF-SAT $\leq_p$ SUBSET-SUM.

Each arrow retains its own premises. HC here is undirected. The TSP complete graph has polynomial size. Subset-sum uses binary target encoding. Optimization TSP and its decision language have different output specifications.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proving a new NP-complete language

1. Specify the decision yes-set, instance parameters, and output.
2. Give a certificate, polynomial length bound, verifier, and acceptance conditions to prove NP membership.
3. Construct a reduction from a known NP-complete source to the new target.
4. Prove source yes implies target yes.
5. Prove target yes implies source yes.
6. Bound construction time, output size, and verification time by polynomials.
7. Handle empty graphs, k=0, repeated literals, disconnected graphs, and unreachable values as applicable.

A suggestive gadget without both implications is not a reduction proof. Converting the new problem back to a known hard problem also has the wrong direction for establishing hardness.

Source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 34.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study